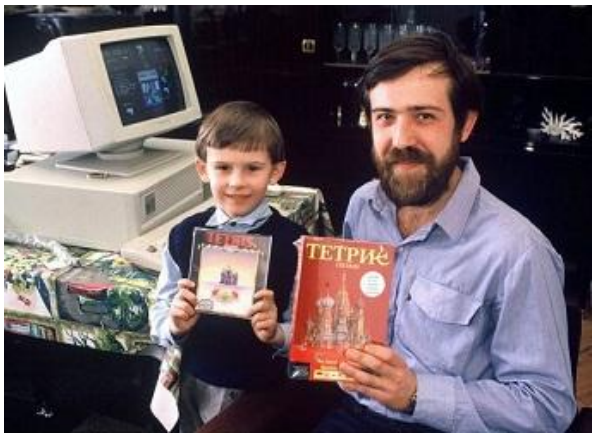
ICS 214 Lab 8

Introduction:



Tetris (as most know) is an addictive puzzle game, that has had many different variations since the 1980's and remains popular today. It is simplistic in its design and embraces our universal desire to create order out of chaos. It was created by Alexey Pajitnov in 1984—the product of Alexey's computer programming experience and his love of puzzles. In the decades to follow, Tetris became one of the most successful and recognizable video games, appearing on nearly every gaming platform available.

If you haven't played it for awhile – have a try before going any further so that you'll have a better understanding of exactly what we are building and to aid with understanding the discussion below: https://tetris.com/play-tetris

This lab is the final Tetris lab in which we put together the building blocks we have been creating in previous labs to build a simple Tetris game. We have we have already completed about 50% of the game in previous labs. This is a **2 week lab.** The amount of coding required to finish it is expected to take at least twice that of a normal lab. There are milestones in the "Demo & Marking" section to help keep you on track. **Do not leave this until the last minute to start on!**

Each one of these objects we have created this far (Point, Tetromino, Gameboard) has been build through composition (as opposed to inheritance). Each object has no knowledge of anything "larger" than itself. A Gameboard has no knowledge of a Tetromino. Instead, it has routines that allow us to set "content" (colors) on a grid. This has been done to keep the relationships between objects as simple as possible.

In this lab we will be writing a class called TetrisGame which is used to contain and co-ordinate how these objects are used together within the context of a game of Tetris.

The end goal is to have basic Tetris gameplay in place – which includes the following:

- Game initialization and setup
- A currentShape (tetromino) on the gameboard.
- A nextShape (or "on-deck") tetromino waiting for the currentShape to be placed (selected randomly).
- Basic tetromino movements for the currentShape (left, right, down, rotate, drop)

- Gameboard functionality – accumulation of blocks, clearing rows (already written)
- Very basic scoring (one point per row completed)
- Game end detection
- Game reset

Objectives:

1. Exposure to using a graphics library (SFML) and namespace.
2. Exposure to the game loop and event driven programming.
3. Exposure to OOP programming with classes (.cpp and .h files)
4. Exposure to project structure & organization.
5. Developing class member functions for re-use
6. "bottom-up" building approach (putting together smaller classes first).

SFML graphics library

The project uses a graphics library called SFML (Simple Fast Multimedia Library) which has multiplatform support (Windows, Linux, macOS). I have put together a tutorial for setting up SFML in Visual Studio 2022 – located in D2L| Content | Labs | Lab8 | SFML Setup.

The focus of this lab is not on about setting up SFML, but it is good to go through the process to show how an external library is added. A tutorial for this has been provided in a separate file. If you follow the directions carefully, it should take no more than 20 minutes to complete. Once a "hello world" project with SFML is set up, a main.cpp will be provided for the Tetris project (which is very similar to the main.cpp that gets created in the SFML tutorial). It just includes some additional code for loading images and delegating loop control to the TetrisGame object.

If you are interested in learning more about SFML, the SFML website has a set of tutorials for learning more. Documentation can be found at: https://www.sfml-dev.org

Multi-platform support:
I have not attempted to set up this project on a Mac or Linux machine though I know other students have successfully done so. If you can get the SFML tutorial set up on whatever platform you're working on, then I expect you should be able to do the project on the same platform. The hardest part is telling your environment how to link to the SFML graphics libraries properly (which for VS2022 was initially a bit of a chore). Tutorials for Mac and Linux can be found at https://www.sfml-dev.org/tutorials/2.5/.

Project Structure

Although there are likely dozens of different ways to go about programming Tetris, heading into a larger project with varying structure or direction would be chaotic difficult to support when it comes to debugging. The given project structure is meant to keep you on track & guarantee completion. One of the things I'm hoping you will notice as you are developing this – is how designing classes, and methods with a singular purpose allows for a lot of reuse. It also provides you with a toolset of routines that can often be used for writing more complex functions and adding further functionality later on.

Remaining Tasks:

Our tasks for the remainder of the project are:

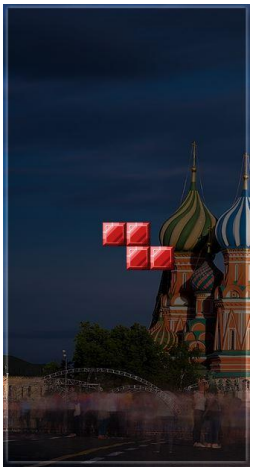
1. Complete GridTetromino.cpp to match GridTetromino.h
2. Create a new project structure for Tetris, include SFML.
3. Copy existing source code into the new project.
4. Develop TetrisGame.cpp to match TetrisGame.h

Develop GridTetromino.cpp

Before we do anything else, let's write the implementation for the GridTetromino class. It is not a large class (40 lines). Since we have a project (Lab 6) already set up for the Tetromino class, let's modify Lab 6 to add this functionality.

GridTetromino.cpp

In the last lab, we discussed our plans for adding functionality to the Tetromino class to give it an awareness of its location on a co-ordinate system. Let's extend the Tetromino class for this purpose and create a GridTetromino class with this added functionality.



Functionality added:

- an x,y location (gridLoc) for the Tetromino on a coordinate system.
- Functions for moving a tetromino (by changing its gridLoc)
- Get a vector of tetromino block locations mapped to the gridLoc.

Additional documentation on this class can be found in the GridTetromino.h.

GridTetromino : public Tetromino
- Point gridLoc
+ GridTetromino() + Point getGridLoc() + void setGridLoc(int x, int y) + void setGridLoc(Point pt) + void move(int xOffset, int yOffset) + std::vector<Point> getBlockLocsMappedToGrid()

The GridTetromino class is the first class that we will develop using inheritance (using Tetromino as the base class). Fortunately, the inheritance used in GridTetromino simply extends the functionality of the Tetromino class without modifying the original functionality. A GridTetromino.h file is given to you. You will need to set up this inheritance when you fill out the GridTetromino class.

All we need to do is to add a GridTetromino.cpp and fill in the methods declared in the header file.

1. Create a backup of your Lab6 source code files (so that you can always refer to the originals if needed). Do not create a copy of the project directory and expect to work from that - Visual Studio seems to have issues with that. Instead, make a backup of your existing Lab6 code files, and continue work on your existing Lab 6.
2. Open up your existing Lab 6, Download GridTetromino.h and the new TestSuite.h to support it from the Project folder on D2L. Add them to the Lab6 project (overwrite the existing TestSuite.h).
3. Complete the method definitions in the .h file using the descriptions given for parameters and taking into account if the methods should be const.
4. Create and complete a GridTetromino.cpp file based on the comments in the GridTetromino.h file.
5. Ensure your GridTetromino class passes the assertion tests in TestSuite.h. Uncomment the following line in TestSuite.h to check your work:

```
#define GRIDTETROMINO
```

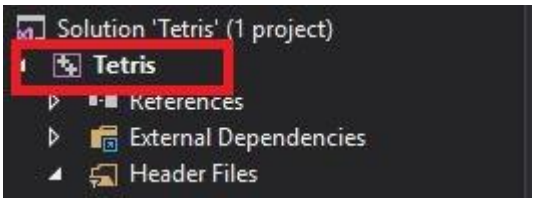
Tetris Project Setup

Step 1: Use the SFML tutorial to setup a project for Tetris

To complete the Tetris project, we will be using a library called SFML (Simple Fast Multimedia Library). There is a document with directions on how to set it up on D2L in: Content | Project | SFML Setup. Follow the instructions to create a new empty project with the SFML libraries included.

Step 2: Copy files into the new Tetris project

Download the following files from D2L and place them in your project folder (the same folder as Source.cpp) To get to that folder, right click on the project name in the Solution Explorer, Open Folder In File Explorer.



Copy all the files listed below into this folder.

- Main.cpp
- tiles.png
- background.png
- RedOctober.ttf
- TestSuite.h
- TestSuite.cpp
- TetrisGame.h

Copy the latest versions of the following files from Labs 6 and 7:

- Point.h
- Point.cpp,
- Tetromino.h
- Tetromino.cpp
- GridTetromino.h
- GridTetromino.cpp
- Gameboard.h
- Gameboard.cpp

Create 2 new folders in your project folder (images and fonts)

- Move (drag) tiles.png and background.png into the images folder
- Move RedOctober.ttf into the fonts folder.

Replace Source.cpp (comment out the whole file) in your Tetris project with Main.cpp

SFML displays images by loading a texture (an image) from a file, and then using a 'sprite' to display that texture in the window. We only use 2 images for Tetris. One for the background, and the other to display the blocks. Have a look at them with an image viewer so you know what to expect.

Add all the .cpp and .h files to your Solution Explorer | Source Files folder (via Add | Existing Item). Warning: I have noticed (with VS2022 particularly) that when I add an existing items, the folder that opens is probably not the one I want! As a result, it is very easy to add a file from a folder you think is the Tetris folder, but it is not. To save yourself the headache of trying to figure out why your files can't find each other, ensure that you are adding existing items from the Tetris folder (and not Lab6 or Lab7 folders).

Drag & drop the .h files (in your Solution Explorer) to the "Header Files" subfolder.

Step 3: Create stub functions to get the project building.

1. Go through the TetrisGame.h file and complete the method declarations (function signatures) with the appropriate return type and parameters (using the documentation given in the file). Keep in mind that any objects that are passed should be done so by reference, and if possible const reference.
2. Create a TetrisGame.cpp file. #include "TetrisGame.h" at the top.
3. Copy over all the function declarations (with comments) from TetrisGame.h to the .cpp file. Make each one into an empty stub function:
   a. Add TetrisGame:: to the beginning of every function name (in TetrisGame.cpp).
   b. Replace the ";" at the end of the function definition with an empty body {}
   c. Put in temporary return values in the following functions requiring a return value (note parameters omitted for brevity):
      - spawnNextShape() { return true; }
      - attemptRotate() { return true; }
      - attemptMove() { return true; }
      - attemptMove() { return true; }
      - isPositionLegal() { return true; }
      - isWithinBorders() { return true; }
   d. Instantiate & initialize the static constants declared in TetrisGame.h to the values specified in the comments. You will need to do this in TetrisGame.cpp. (Google: learncpp static member variables if you forget how).

Edit the TetrisGame constructor to do the following (ie: don't complete the whole constructor):

- Set the formal parameters passed in to the corresponding 4 member variables of TetrisGame (the names should match). Two of them are references and so they will need to be initialized (instead of just assigning them).

Try building and running. If everything worked, you should get a game screen with a white background on it (in addition to the console output).

A word about the two new files…

Main.cpp

This file is given to you, and so you won't have to change it (much) – but you should be able to understand everything that is going on inside it. It is very similar to the Main.cpp that we will create in the SFML setup tutorial (with some slight modifications).

It is responsible for the following:

- Running some assertions to do some basic testing on the classes.
- Creating the game window
- Loading in images
- Instantiating a Tetris game
- Running the main game loop
- Detecting keypress and/or window events
- Keeping track of time
-

Take a moment and look through Main.cpp. This is the code that starts everything in motion. Some of the objects instantiated in Main.cpp are SFML objects. Notice that they use the namespace "sf::". It uses Textures and Sprites from the SFML library to load and display graphics. It runs the main game loop and sends input events and info regarding time tracking to the TetrisGame class so that TetrisGame can determine what to do with that information.

TetrisGame.h

The TetrisGame class is responsible for:

- Setup and management of the gameboard
- Managing the falling tetromino (currentShape) and its movement/rotation.
- Managing the "on-deck" tetromino (nextShape)
- Drawing game elements to the screen
- Responding to keyboard input and affecting the falling tetromino



This is the class that "runs the show" when it comes to the Tetris game. It was designed so that if you wanted to, you could instantiate two Tetris games running simultaneously side-by side (multi-player). The TetrisGame class is an encapsulation of everything needed to manage a single Tetris game.

Additional documentation on this class can be found in TetrisGame.h.

Displaying the Gameboard

In Main.cpp, Uncomment the 5 lines that declare, setup, and draw the background using the backgroundTexture and backgroundSprite variables. The background image gets loaded into a sf::Texture object (from disk), and the texture object is used to create a sf::Sprite object. This sprite object then gets drawn to the screen every single game loop. If you run the game now, you should now see the background displayed.

Notice that on the line directly after the backgroundSprite is drawn, the TetrisGame object instance is told to draw(). Currently this method is empty, but this will eventually be used to draw our Gameboard and Tetromino objects.

Developing TetrisGame.cpp

At this point we have one last class to develop: the TetrisGame Class. This class will act as a container class for all the components of the game, as well as defining the behaviours that make them work together within the context of the game.

TetrisGame will be responsible for:

- creating/initializing components (gameboard, currentShape, nextShape),
- gameplay logic (tetromino movement/rotation, boundaries, copying content to the gameboard),
- drawing everything in the game (gameboard, currentShape, nextShape),
- event handling (keyboard input).

Give yourself a pat on the back for getting this far! This next class will be our largest and most involved. Fortunately, because we have abstracted a lot of the functionality into components (Point, GridTetromino, Gameboard) we should be able to write a class that focuses on the bigger picture of the actual game mechanics.

The development for TetrisGame.cpp is broken into tasks which should hopefully allow you to follow a logical progression, and still be able to compile and test your code along the way. Let's get started!

Draw a block

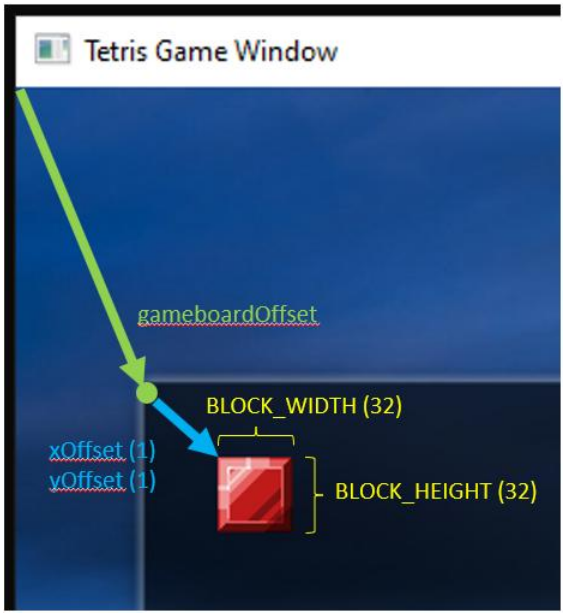
Fill in drawBlock() (using comments in .h file). Reference this SFML website link to help understand how setTextureRect(), IntRect, setPosition(), and window.draw() work: www.sfml-dev.org/tutorials/2.5/graphics-sprite.php. You will need to call setTextureRect() and pass it an instance of type of a SFML object: sf::IntRect(x,y,width,height). This will define a rectangle which will tell the SetTextureRect() function what portion of tiles.png that you want to draw. Start with sf::IntRect(0, 0, BLOCK_WIDTH, BLOCK_HEIGHT) and develop from there.



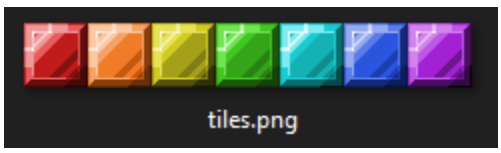
The TetrisGame.draw() method is one that we define to do all the drawing for the game. It is called (from main()) **every single game loop** to redraw our game components. So, when you think you have drawBlock() figured out – test it by temporarily calling drawBlock() from TetrisGame.draw(). Use Point(0,0) for topLeft, and 0's for X and Y. You should get the following visual.

The gameboardOffset member variable of TetrisGame contains an offset of the top left position of where the gameboard should appear. Try using this offset with drawBlock() to shift the red square to the top left of the gameboard in the window.

drawBlock() also takes an xOffset and yOffset parameter. Try passing in: 1 and 1. If drawBlock() takes the xOffset and yOffset into account when determining where to draw the block, it should produce the following result.



You will next need to modify setTextureRect() within drawBlock() to allow you to pass in a particular TetColor and then have it adjust the IntRect to select the appropriate colored block (as a texture). . Note that the color enums match the order of the colored blocks in the graphics file), which should allow you to static cast a color and figure out the offset (in pixels) within the file.



Change the color passed to drawBlock() and see if you can get the color you want to appear. Details on setTextureRect() in the the SFML link above should help with this. Once you get this working, you can remove the drawBlock() call in draw().

Draw the gameboard



Next, fill in the drawGameboard() method to draw every block in the board. Put in a temporary TetColor::BLUE_DARK into the call to drawBlock(). Add a call to drawGameboard() inside draw(). Test it out. You should see your gameboard filled with blue blocks!

If you got a different color than you were expecting, you missed a step. You need to modify drawBlock() to calculate its starting offset to the block that you want to draw based on the color specified (view tiles.png)

If the gameboard is displayed somewhere other than exactly where it should be, you need to check the offset passed to drawBlock() is the gameboardOffset point as its topLeft.



Once drawGameboard() is working, modify it to draw the color based on the board's contents at the current x,y being examined (only call drawBlock() if board.getContent(x,y) is not Gameboard::EMPTY_BLOCK). Run it again, and the whole board should be empty (since our gameboard gets initialized to empty). To test it works correctly, temporarily set the content of the first block to a color: board.setContent(5,5,2), and check if the gameboard shows the content shown here.

Generate a random shape

Write a public static function for the Tetromino class:

static TetShape Tetromino::getRandomShape()

Have this function return a random TetShape. Hint: if you add an enumeration to the TetShape enums (in the last spot) called COUNT – you can then use it with the rand() function (using the mod operator) to get a random shape (cast TetShapeCount to an int, and then cast the result back to a TetShape before returning). You will need to seed the randomizer in main().

Display a shape on the gameboard

Next, let's see if we can get a Tetromino being displayed. In the TetrisGame constructor (temporarily) set currentShape to a random shape (using Tetromino::getRandomShape()). There is a Point member variable in Gameboard.h called spawnLoc (a block XY location from which our tetromino should spawn). Write a public accessor for it called getSpawnLoc(). Set the grid location of the current shape using this function.

In our draw() method, make a call to drawTetromino(), pass it our currentShape, and use the gameboardOffset as the topLeft parameter.



Fill in drawTetromino() using the comments as a guide.

Test it out – you should see a randomly generated tetromino at the top of the gameboard.

You may notice at this point that your Tetromino is upside down from its original orientation. This is because the Tetromino's positive Y direction is up, but the Gameboard's positive y direction is down. This should not cause any issues at all. Why has this been done? Because the gameboard co-ordinate system mirrors the way that the screen resolution itself works [0,0] at the top left – so when it comes time to draw the gameboard on the screen, it is *much* easier to deal with.

Add shape rotation and movement

Let's implement our rotation functionality!

Fill in attemptRotate()

Modify TetrisGame.onKeyPressed() to switch on event.key.code. Have it call attemptRotate() when it detects a sf::Keyboard::Up event. Test it out. It should rotate when you press the up arrow on the keyboard!

Fill out attemptMove() in the same way. Modify onKeyPressed() to call attemptMove() to move left, right, and down.

Test it out. Notice that the shape has no restrictions on where in can move.

Add limitations on shape movement

Let's fill in isPositionLegal() to fix this. It makes a call to another method:

- isWithinBorders()

Let's fill in this as well. Test after you complete it. We can easily test if the border restriction works by moving to the edges. To test if we are prevented from intersecting locked blocks, let's put a locked block right in the center of the gameboard (for the sake of testing).

In the TetrisGame constructor, specify:

board.setContent(Gameboard::MAX_X/2, Gameboard::MAX_Y/2, 1);

Finish this off by using Gameboard's areAllLocsEmpty() method and passing it the shape's mapped locs to test if the moved shape collides with any locked blocks on the gameboard.

Add drop functionality

Let's fill in our drop() and lock() functions next. Call them both when the user hits a spacebar. Test that these work by dropping a shape to the bottom of the gameboard as well as on top of the block we placed in the center. This can be accomplished in a single line using a loop and the attemptMove() function. Once this has been tested, we can remove the extra block we set on the screen.

Set up ticking functionality to trigger movement

Let's look at processGameLoop() next:

This method is called from the infinite game loop set up inside our main() function that continues until the user closes the window.

We need to have processGameLoop() somehow trigger a call to tick() which is called approximately every half second (the actual length of time is stored in the member variable secsPerTick).

The processGameLoop() function is called many times a second. The main() function calculates elapsed time every game loop, and passes this to processGameLoop(secondsSinceLastLoop) that will tell us the fraction of a second since the last call to it was made. If we add this number to a total (secondsSinceLastTick), we can determine when it is time to call the tick() function (by comparing to secsPerTick). When secondsSinceLastTick is larger than secsPerTick, call tick() and decrement secondsSinceLastTick by secsPerTick (the result should always be larger than 0).

Test this out – put a cout statement in tick() and ensure it is called every half second or so. Once you've verified it works, fill out the tick() function based on its comments.

Test that it works, you should be able to see a shape moving down the gameboard.

Handle shape "locking" (copying a shape to the gameboard)

Fill out lock() according to the comments, and test to ensure a tetromino is immoveable after you drop it. Lock should be called in 3 situations:

- After a drop()
- When tick() attempts to move a shape down and fails
- When the player attempts move a shape down (via the keyboard) and fails

Make sure each one of these is handled correctly.

Handle the initial game start (via reset):

When we added code to our TetrisGame constructor to get a currentShape set up, we were 'side-stepping' the regular setup for the purpose of getting things working. It's time to go back and implement a proper setup.

In TetrisGame, we have two GridTetromino members:

1. currentShape (the shape actually moving)
2. nextShape (the shape that is 'on deck' or next up)

Fill in pickNextShape() and spawnNextShape() (using the comments as a guide)

Fill in reset(). This will require calling 2 functions that currently have empty implementations. That's ok for now. What we're interested in is getting our nextShape set up.

Now go to the TetrisGame constructor, remove the lines that set the currentShape and its location. Replace them with a call to reset().

Test to ensure that the game still starts properly.

Handle logic after shape placement:

Let's fill in the rest of processGameLoop() and make it so the game knows what to do once a shape has been placed. Check shapePlacedSinceLastGameLoop. If true, we need to take a few actions, but first – set the shapePlacedSinceLastGameLoop flag to false to indicate that we've handled a new placement (which we're about to do). Next, attempt to spawn the next shape.

If spawnNextShape() succeeds, we need to:

- Pick a new next shape
- Remove any completed rows from the board
- Set & update the score (google: "scoring for tetris" for the original scoring system)
- determineSecondsPerTick() (as the score increases, we may want to change the seconds per tick to be less – making the tick rate faster)

If spawnNextShape() fails:

- it means that the gameboard was full and that spawning a new shape resulted in an illegal placement – in which case, simply call reset() to start the game over.

Test it out. Your game should be playable at this point!

Display the next shape:

Modify draw(). Add a line to draw the nextShape. Use the nextShapeOffset as the origin.

Displaying the score:

To draw the score, we first need to load a font, and set the scoreText to use it. To do this, add the following lines to the TetrisGame constructor:

```
// setup our font for drawing the score
if (!scoreFont.loadFromFile("fonts/RedOctober.ttf"))
{
	assert(false && "Missing font: RedOctober.ttf");
```

```
};
scoreText.setFont(scoreFont);
scoreText.setCharacterSize(18);
scoreText.setFillColor(sf::Color::White);
scoreText.setPosition(425, 325);
```

In the draw() function, we can then add:

```
window.draw(scoreText);
```

Lastly, fill in the updateScoreDisplay() function (see comments in .h file) and ensure it is called whenever the score changes.

Testing:

The best way to find errors is to play it. Test it to ensure it works as expected. Have a classmate play your version and try to break it (they may try things to break it that you never thought of). Play test someone else's game to see if it works the same as yours or if there is anything different.

That's it… That's all… You're done (provided everything works)! Congratulations!! Go watch this video to celebrate all your hard work: https://www.youtube.com/watch?v=zH_omFPqMO4 .

Disclaimer: the code written in the video is full of magic numbers and is mostly written in the main function. There is little to no modularity or encapsulation so I wouldn't even consider it OOP. The code you have just written is far easier to understand and extend.

There is a list of ideas for further (optional) development on the last page in case you're extra keen and want to develop more for fun.

Milestones & Marking

You have 2 weeks to complete this lab. Completion of this lab is worth twice as much as a regular lab. You will have to manage your own time with this one. There will not be a demo at the end of each week. Use the milestones below to help you keep on track.

Milestones: As a rough guide:

Week 1: Complete GridTetromino and set up the project in SFML. You should have the ability to draw the gameboard using SFML and fill it with blocks. You can display a Tetromino on the screen. You should be able to move and rotate the Tetromino.

Week 2: Add in boundaries for Tetromino movement. Detect collision with a stationary block on the gameboard. Add in drop() functionality. Add tick() functionality to move the tetromino down the screen automatically. Locking (copying tetromino blocks to the gameboard), respawning a tetromino, creating an "on deck" tetromino, logic to remove completed rows, scoring, and game end/restart.

This is not meant to tell you to stop once you have reached the suggested goal for the week. It is meant to let you know you are falling behind if you haven't completed the week's goals. By all means – finish it early if you can before things get too busy with other classes!

To demo your project, your code must pass all the TestSuite.h tests. There are no additional tests for the TetrisGame class – that portion is tested by playing the game.

- Run the TestSuite tests in main() this to prove to yourself that all your class member functions work as expected. If they don't, track down the errors and fix them.

Marking: [out of 20]

Marking will be done via a demo, a code walkthrough, and verification that TestSuite.h runs without any issues.

- Does it work as expected? (Are there bugs/crashes?)
- Have objects that get passed as parameters been passed by reference? Are they const when possible?
- Variables: Have good variable & const names been used?
- Are variables in local scope instead of global?
- Are there comments to explain anything non-trivial or not obvious? Is there a comment at the top of every function?
- Have variables or constants been used instead of "magic numbers" in the code.
- Have asserts been added to functions to check for valid parameters?

Ideas for further (optional) development

The intention of this lab was to get the basic gameplay of Tetris working.

There are a lot of additions that could be made to develop the game further. Here are some possibilities in case you get done and you're keen to keep experimenting.

- Adding music & sound effects. The are some music/sfx available in the Lab8 folder. (the online SFML documentation will tell you how to play sounds)
- Adding a 'ghost' tetromino to show the place where the currentShape would land if it was dropped at any point. This is actually surprisingly easy! Here's how to do it:
    - Add a new shape called ghostShape
    - Add a TetrisGame function called updateGhostShape() and in it:
        - copy currentShape into ghostShape
        - drop the ghostShape
    - add a drawTetromino(ghostShape) call to the TetrisGame.draw() method.
    - after every attemptMove() or attemptRotate(), call updateGhostShape()
    - add an additional parameter to drawBlock() and drawTetromino: float alpha = 1.0f
    - use the sprite.setColor() function to set the alpha to be semi transparent: www.sfml-dev.org/tutorials/2.5/graphics-sprite.php, and use lower alpha of 0.5f when making the drawTetromino(ghostShape, 0.5f) call.
- Write a better algorithm for selecting random shapes. (fill a vector with 2 of every shape, shuffle it, keep pulling off shapes, when the vector is empty, refill and shuffle). This way there is never a "dry spell" for to long, and we never randomly get too many of one type of shape.
- Making the tick rate faster as you reach higher scores. The game is already set up for this – the logic just needs to be added to determineSecsPerTick()
- Add in a short delay when the player completes a line so the player has a chance to recognize the fact that a line is about to disappear.